BPDTE: Batch Private Decision Tree Evaluation via Amortized Efficient Private Comparison

Huiqiang Liang[1,4], Haining Lu[2], Yifeng Guo[3], Geng Wang[2], Haining Yu[1], Hongli Zhang[1], Baoyu An[3], Jinyu Li[3], and Li Su[3] *

[1] Harbin Institute of Technology, China
24b903126@stu.hit.edu.cn,{yuhaining, zhanghongli}@hit.edu.cn
[2] Shanghai Jiao Tong University, China
{hnlu, wanggeng}@sjtu.edu.cn
[3] China Mobile Information System Integration Co., Ltd, China
{guoyifeng, anbaoyu, lijinyu, suli}@cmict.chinamobile.com
[4] ViewSouces (Shanghai) Technology Co., Ltd, China

Abstract. Machine learning as a service requires clients to entrust their information to the server, raising privacy concerns. Private Decision Tree Evaluation (PDTE) are proposal to address these concerns in decision trees, which are fundamental models in machine learning. However, existing solutions perform poorly with massive datasets in real-world applications, therefore, we focus on the batching variant called BPDTE, which supports a single evaluation on multiple data and significantly improves performance. Firstly, we propose three private comparison (PrivCMP) algorithms that privately compare two numbers to determine which one is larger, by utilizing thermometer encoding and a novel folklore-inspired dichotomy method. These algorithms are non-interactive, batchable, and high-precision, achieving an amortized cost of less than 1 ms at 32-bit precision. Secondly, we propose six BPDTE schemes based on our PrivCMP and the Clear Rows Relation (CRR) algorithm, introduced to ensure batching security. Experimental results show that our schemes improve amortized efficiency, offer more flexible batch sizes, and achieve higher precision. Finally, we provide a formal security analysis of these schemes.

1 Introduction

Machine learning has been widely adopted and performs well in services such as healthcare, financial services, and data classification [4, 35, 40, 43, 44, 46]. Typically, users are required to upload their sensitive information to service providers, so they may be concerned about their private data being illegally collected, potentially leading them to abandon such services. Conversely, service providers must protect their proprietary information to prevent leaks that could lead to unauthorized distribution or misuse of their services.

Decision trees [30,33], as a fundamental classification model in machine learning, are widely used due to their simplicity, interpretability, and ease of training. In decision tree evaluation, a vector of numbers and a decision tree are input, and a classification result is output, with at least one party uploading their input to another. To enhance privacy and security, PDTE has been proposed in recent research [5, 14, 15, 26]. PDTE allows a client to use the server's decision tree to obtain classification results without revealing their own input, while ensuring that the server discloses no knowledge of the decision tree. During the evaluation, at each decision node, it must determine whether to proceed to the left or right child node while maintaining privacy. PrivCMP aims to compare two values provided by different users and privately determine which one is larger, making it a core component of PDTE. Similar to the millionaire's problem [42], numerous techniques are available for solutions [17,20,31,39], including oblivious transfer, homomorphic encryption, and special encoding. Due to the nonlinearity and atomicity of PrivCMP, various applications are employed, such as private data access, secure sorting, auctions, and federated learning.

Existing PDTE include both interactive schemes [6,23,27,45] and non-interactive schemes [1,5,14,15,28,38]. Interactive schemes typically rely on secure multiparty computation, requiring higher bandwidth and imposing simultaneous online requirements for both the client and server. Non-interactive schemes are primarily based on leveled homomorphic encryption (LHE) [7,19] or fully homomorphic encryption (FHE) [13], demanding greater computational resources and being constrained by circuit depth or heavy bootstrapping. There are several practical non-interactive schemes, such as Sortinghat [14], XXCMP-PDTE [2], and RCC-PDTE [2], which take a few seconds for a single evaluation. These schemes are fast enough for small datasets, but not efficient for handling millions of records. BPDTE_CW [15] designs a batching process for multiple data points, but the batch size is fixed at 16,384, resulting in significant communication overhead. Additionally, Sortinghat and XXCMP-PDTE offer 12-bit and 36-bit precision, respectively, while RCC-PDTE provides 128-bit precision with a high computational cost. BPDTE_CW supports high precision, but the communication cost increases significantly with precision.

In this paper, we focus on private comparison and decision tree evaluation for large datasets, enhancing server-side computing performance on client-encrypted data. We propose three PrivCMP algorithms and six BPDTE schemes that are non-interactive, batchable, high-precision and efficient. Our contributions are outlined below.

* This work was funded by the Harbin Institute of Technology-China Mobile Communications Group Co., Ltd. 5G Application Innovation Joint Institute. (Corresponding author: Haining Yu.)

- We design three PrivCMP algorithms with a simpler structure and high precision. Their precision increases exponentially with multiplicative depth due to our thermometer encoding and a new folklore-inspired dichotomy method. These algorithms support two ciphertext inputs, and the output can directly compute Hadamard addition and multiplication. By encoding multiple numbers into a single input, our algorithms allow perform a single comparison on multiple data points.
- We propose six BPDTE schemes that combine our three PrivCMP algorithms with two SumPath variants. A new CRR algorithm are designed for Level Up's [2] SumPath to ensure batching security, obscuring the positions and value relations in multiple data inputs with logarithmic random permutation.
- We benchmark our PrivCMP and BPDTE schemes across various decision trees and data precisions. The results demonstrate that our PrivCMP is both computationally and communication-efficient. The BPDTE schemes outperform [2,14] in terms of efficiency and offers more flexible batch sizes and low communication cost compared to the state-of-the-art.

Section 2 provides an overview of related work. Section 3 introduces the fundamental concepts. Section 4 outlines the system model and problem definition. In Section 5, we present thermometer encoding and the dichotomy method, followed by the design of our PrivCMP. Section 6 introduces our BPDTE schemes. Section 7 presents the security analysis and proof. Section 8 benchmarks the performance of our schemes. Finally, Section 9 provides an conclusion for our work.

2 Related Work

PDTE can be considered as a two-party secure computation. Several works, such as [6, 8, 23, 27], have been proposed with interaction, employing techniques like secret sharing, garbled circuits, and oblivious transfer. For example, Bost et al. [31] used multivariate polynomials to obtain classification results. Tueno et al. [38] employed Oblivious RAM, which requires multiple rounds communication. Liu et al. [26] trained and evaluated models on sensitive data using function secret sharing. Additionally, Tai et al. [36] constructed the PDTE with the PrivCMP improved in [16, 41], but the comparison results must be decrypted before further computations. Nateghizad et al. [29] used the DGK and Paillier cryptosystems, which also require decryption of the comparison output, introducing additional interaction.

Generally, secure multi-party computation methods require multiple rounds of interaction, resulting in significant communication overhead. In real-world scenarios, many prefer non-interactive approaches for implementing the PDTE scheme to enhance performance. The non-interactive PDTE requires its key component, PrivCMP, to be non-interactive as well. XCMP [28] encodes numbers into a polynomial and computes the comparison results using only one polynomial multiplication, making it fast and non-interactive. However, the comparison result lacks expressiveness, as it can only perform polynomial addition and scalar multiplication, limiting its utility for subsequent computations. Additionally, the precision of XCMP is constrained by the degree of the polynomial, and its One-Hot encoding method introduces significant communication overhead. We demonstrate the advantages of our PrivCMP compared to others in Table 1.

Table 1. private comparison

Methods	Bit precision	Output expressive
XCMP [28]	16	Polynomial add, Scalar mult.
SortingHat [14]	12	Polynomial add, Scalar mult.
XXCMP [2]	36	Polynomial add, Scalar mult.
RCC [2]	287	Hadamard add, Hadamard mult.
FOLKLORE [2]	1024	Hadamard add, Hadamard mult.
CWCMP [15]	> 256	Hadamard add, Hadamard mult.
TECMP(ours)	26624	Hadamard add, Hadamard mult.
RDCMP(ours)	1024	Hadamard add, Hadamard mult.
CDCMP(ours)	1024	Hadamard add, Hadamard mult.

Subsequently, PDT-BIN and PDT-INT by Tueno et al. [37] instantiated PrivCMP from [11, 12] and [24]. These schemes are non-interactive and secure, but their overall computational cost remains high due to the inefficiency of the FHE and tree traversal algorithms. Sortinghat [14] uses the PolyComp algorithm, an improved version of XCMP [28], which offers low computation cost but is limited by comparison precision. Additionally, PolyComp's comparison results are RLWE ciphertexts, which require conversion [10] to RGSW ciphertexts for subsequent computations. This makes PolyComp inflexible for PDTE and less effective in high-precision scenarios. Another approach in [14], based on gate circuits in FHE, supports arbitrary precision and trees, but the overall cost remains impractical for applications. PROBONITE [5] also employs XCMP, with the defining feature of using SinglePath in tree traversal, where each layer uses blind rotation to locate the comparison node, following the typical process for evaluating decision trees on plaintext. However, confirming the comparison node at each layer incurs considerable overhead, and the computation cost still requires thorough benchmarking to assess its overall efficiency.

Recently, Level Up [2] introduced an extended precision version of XCMP [28], called XXCMP, which performs additional operations to extend precision as outlined in [3] and requires at least one plaintext input. The RCC algorithm builds on methods from [21, 34], and [32], with further improvements such as batch ability and support for two ciphertext inputs. Their PDTE approach achieves a high level of efficiency in both computation and communication costs, except for realizing the batching process. Subsequently, BPDTE_RCC and BPDTE_CW were introduced by Cong et al. [15], improving the RCC and CW encoding methods from Level Up [2] through piece-by-piece operations, leading to RCCMP and CWCMP (originally referred to as the batched range cover comparison comparator and constant-weight piece-wise comparator, respectively). Furthermore, they enhanced the SumPath method by introducing an adapted version that maps values from $\{1, \ldots, d\}$ to 1 and $\{0\}$ to 0. This adjustment significantly reduces the communication cost of PDTE, though it sacrifices some computational efficiency. They realize PDTE with batching, but the batch size is a large constant, making it inefficient for small batch sizes.

3 Preliminaries

Table 2 standardizes all symbols used throughout this paper. The function $b \leftarrow \mathrm{I}(\mathit{statement})$ that $b = 1$ if *statement* is true and $b = 0$ otherwise. The function $\mathrm{len}(\boldsymbol{x})$ that input a vector $\boldsymbol{x}$ and output the length of $\boldsymbol{x}$.

Table 2. Summary of notation

Notation	Description
λ	security parameter
L	arithmetic circuit depth
$\mathbb{M}$	message space
$\mathcal{C}$	arithmetic circuit space
N	polynomial degree
p	plaintext modulus
q	ciphertext modulus
R_q	polynomial ring with modulo q
Z_p^N	integers ring with modulo p in dimension N
Rotate_k	the k slots left rotate
$[x]$	$\{0, 1, \ldots, x-1\}$
$\mathcal{M}$	decision tree
$\mathcal{N}$	the nodes set of the decision tree
a	the attribute index of the node
t	the threshold of the node
v	the leaf node value

3.1 Leveled Homomorphic Encryption

FHE allows any arithmetic circuit to be performed on ciphertext in the same manner as on plaintext, but heavy bootstrapping results in slower performance. LHE avoids bootstrapping, thereby achieving higher computational efficiency. However, it is limited in executing arbitrary arithmetic circuits because each operation in LHE increases the noise within the ciphertext, and decryption will fail when this noise exceeds a certain threshold. Arithmetic circuits consist of combinations of multiplication, addition, and rotation. The noise growth is exponential during the multiplication of two ciphertexts, while other operations grow linearly. Therefore, the 'level' of LHE is primarily determined by the depth of ciphertext multiplications.

A public key scheme of LHE [7, 19] consists of four probabilistic polynomial-time algorithms, as follows: $\mathrm{LHE} = (\mathrm{KeyGen}, \mathrm{Enc}, \mathrm{Dec}, \mathrm{Eval})$.

- $\mathrm{KeyGen}(1^\lambda, 1^L) \rightarrow (pk, sk, evk)$: Take as input the security parameter and the depth of arithmetic circuits, and output a public key pk, a secret key sk, and an evaluation key evk.
- $\mathrm{Enc}(pk, \eta) \rightarrow c$: Take as input the pk and a message $\eta \in \mathbb{M}$, and output a ciphertext c.
- $\mathrm{Dec}(sk, c) \rightarrow \eta\prime$: Take as input the sk and ciphertext c, and output a message η'.
- $\mathrm{Eval}(evk, \mathrm{f}, c_1, c_2, \ldots, c_l) \rightarrow c_\mathrm{f}$: Take as input the evaluation key evk, a function $\mathrm{f} : \mathbb{M}^l \rightarrow \mathbb{M}$ with depth-L arithmetic circuits, and $c_1, c_2, \ldots, c_l$, and output a ciphertext c_f.

Definition 1. (Correctness). Let $\mathcal{C} = \mathcal{C}_\lambda$ be a class of functions with depth-L arithmetic circuits. A LHE scheme is correct if, for any $\mathrm{f_k} \in \mathcal{C}$ and any inputs $\eta_1, \ldots, \eta_l \in \mathbb{M}$, it holds that

$$\mathrm{Dec}(sk, \mathrm{Eval}(evk, \mathrm{f}, c_1, \ldots, c_l)) = \mathrm{f}(\eta_1, \ldots, \eta_l) \tag{1}$$

where $c_i \leftarrow \mathrm{Enc}(sk, \eta_i)$, $(pk, sk, evk) \leftarrow \mathrm{KeyGen}(1^\lambda, 1^L)$.

Definition 2. (Semantically secure [25]): A LHE scheme is semantically secure if, for every polynomial-time adversary $\mathcal{A}$, there exists a polynomial-time simulator $\mathcal{S}$ such that for every $x \in \mathbb{M}$, it holds that

$$|Pr[\mathcal{A}(pk, evk, \mathrm{Enc}(sk, x)) = 1] - Pr[\mathcal{S}(pk, evk) = 1]| < \mathrm{negl}(\lambda) \tag{2}$$

where $(pk, sk, evk) \leftarrow \mathrm{KeyGen}(1^\lambda, 1^L)$.

An LHE scheme based on ring learning with errors (RLWE), where the plaintext space $\mathbb{M} = R_p$, and the corresponding ciphertext space are R_q^2. The Chinese Remainder Theorem can realize the LHE scheme with batching. In batching operation, the corresponding plaintext is represented as $\mathbb{Z}_p^{2\times N/2}$, the $\boldsymbol{x}$ is shown below, and $\boldsymbol{y}$ is the same.

$$\boldsymbol{x} = \begin{bmatrix} \boldsymbol{x_a} \\ \boldsymbol{x_b} \end{bmatrix} = \begin{bmatrix} x_{a,0}, x_{a,1}, \ldots, x_{a,N/2-1} \\ x_{b,0}, x_{b,1}, \ldots, x_{b,N/2-1} \end{bmatrix}, x_{a,i} \in Z_p \tag{3}$$

Then, the addition and multiplication in the ciphertext space are Hadamard addition and multiplication in plaintext space.

$$\boldsymbol{x} + \boldsymbol{y} = \begin{bmatrix} x_{a,0} + y_{a,0}, x_{a,1} + y_{a,1}, \ldots, x_{a,N/2-1} + y_{a,N/2-1} \\ x_{b,0} + y_{b,0}, x_{b,1} + y_{b,1}, \ldots, x_{b,N/2-1} + y_{b,N/2-1} \end{bmatrix} \tag{4}$$

$$\boldsymbol{x} \cdot \boldsymbol{y} = \begin{bmatrix} x_{a,0} \cdot y_{a,0}, x_{a,1} \cdot y_{a,1}, \ldots, x_{a,N/2-1} \cdot y_{a,N/2-1} \\ x_{b,0} \cdot y_{b,0}, x_{b,1} \cdot y_{b,1}, \ldots, x_{b,N/2-1} \cdot y_{b,N/2-1} \end{bmatrix} \tag{5}$$

And the rotation in the plaintext space is defined as follows.

$$\mathrm{Rotate}_k(\boldsymbol{x}) = \begin{bmatrix} x_{a,k}, x_{a,k+1}, \ldots, x_{a,N/2-1}, x_{a,0}, \ldots, x_{a,k-1} \\ x_{b,k}, x_{b,k+1}, \ldots, x_{b,N/2-1}, x_{b,0}, \ldots, x_{b,k-1} \end{bmatrix} \tag{6}$$

3.2 Private decision tree evaluation

PDTE involves three roles: evaluator, model owner, and data owner. When inputting attribute vectors and a decision tree, the evaluator starts at the root node, assigning state values to the child nodes based on the comparison results at each internal node, and then derives the classification result from a specific leaf node. Following the work of [23], PDTE is divided into three components: node comparison, node selection, and tree traversal. The tree structure is shown in Figure 1.

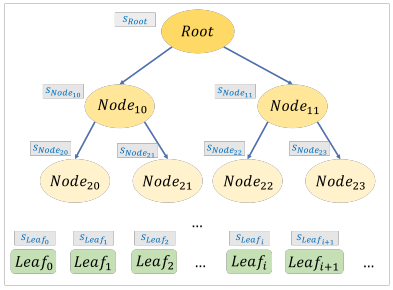


Fig. 1. The tree consists of nodes and leaves, with the root also being a node. Each of these nodes and leaves has a state value for the tree traversal.

Node comparison is the core building block. The evaluator identifies the threshold t and index i from the note and retrieves the attribute vector $\boldsymbol{x}$ from the data party. It then obtains the attribute $x[i]$ in attribute vector. Next, the PrivCMP takes over and outputs the comparison result $b \leftarrow \mathrm{I}(x[i] > t)$. In node selection, the state value of the node is determined by the comparison result from its parent, with the state value of the left and right children being inverses of each other. For example, if the parent's comparison result $b \in \{0, 1\}$, the state value of the left children is b and the right is $\neg b$.

After computing the state values of all nodes, tree traversal uses these values to determine the final leaf. There are three methods for this process: SumPath, MulPath [23], and SinglePath [5].

- SumPath: $s_{leaf_i} = \sum_{node_j \in \{root \to \cdots \to leaf_i\}} s_{node_j}$
- MulPath: $s_{leaf_i} = \prod_{node_j \in \{root \to \cdots \to leaf_i\}} s_{node_j}$
- SinglePath: $root \to node_{1*} \to node_{2*} \to \cdots \to leaf_i$

SumPath sums the state values along each path from the root to the leaves, updating the leaf nodes with the resulting summation. MulPath multiplies the state values along each path from the root to the leaves, updating the leaf nodes with the resulting product. SinglePath performs one node comparison at each layer, like the normal decision tree traversal.

4 Problem formulation

4.1 System model

We focus on BPDTE and PrivCMP, similar to Single Instruction Multiple Data (SIMD), achieving single evaluation on multiple data. BPDTE has two parties: the client and the server. The client acts as the data owner, holding the attribute vectors, while the server serves as both the model owner and evaluator, holding the decision tree and performing the evaluation. Firstly, the client encrypts the attribute vectors and sends its ciphertexts to the server. Second, the server executes the BPDTE scheme on the decision tree and the client ciphertexts, outputs the ciphertexts of classification result, and sends them to the client. Third, the client decrypts the server's ciphertexts and obtains the classification results. Figure 2 shows the interactive model.

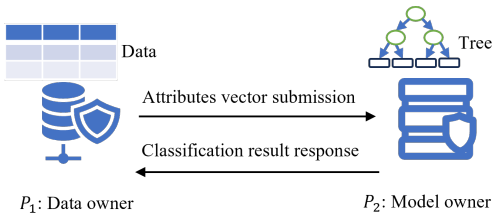


Fig. 2. BPDTE overview, there are two parties, P_1 has a attribute vectors $\boldsymbol{x} = \{\boldsymbol{x_0}, \boldsymbol{x_1}, \dots\}$ and P_2 has a decision tree $\mathcal{M}$, after finished the protocol, P_1 learn the output $\boldsymbol{v}^* \leftarrow \{\mathcal{M}(\boldsymbol{x_0}), \mathcal{M}(\boldsymbol{x_1}), \dots\}$ and nothing else, P_2 learn nothing.

PrivCMP follows the same process, involving two parties and their private input vectors: one party receives the comparison results, while the other party obtains nothing. We first design the secure PrivCMP, then construct our BPDTE.

4.2 Problem defination

The client and server are both semi-honest, meaning they strictly follow the protocol instructions but aim to learn additional information from the received messages. In the simulation paradigm, a protocol is considered computationally secure if a simulator exists that, given only the input and output of the party, can generate a view computationally indistinguishable from the party's real view. We formalize the ideal functionality below, then design our scheme and prove that it realizes this ideal functionality to demonstrate security.

Ideal PrivCMP functionality: P_1 inputs a private vector $\boldsymbol{x}$, and P_2 inputs a private vectors $\boldsymbol{t}$. The output is the vector $\{\mathrm{I}(x[i] > t[i]), i \in [\mathrm{len}(\boldsymbol{x})]\}$, which is revealed to P_1, while P_2 learns nothing.

Ideal BPDTE functionality: P_1 inputs a private attribute vector $\boldsymbol{x} = [\boldsymbol{x}[\boldsymbol{0}], \dots, \boldsymbol{x}[\boldsymbol{M-1}]]^T$, and P_2 inputs a private tree $\mathcal{M}$. The output is $\boldsymbol{v} = [\mathcal{M}(\boldsymbol{x}[\boldsymbol{0}]), \dots, \mathcal{M}(\boldsymbol{x}[\boldsymbol{M-1}])]$, which is revealed to P_1, while P_2 learns nothing.

Ideal CRR functionality: P_1 inputs a private vector $(M, L, \boldsymbol{x}, \boldsymbol{y})$. The output is $(M, L, \boldsymbol{x}', \boldsymbol{y}')$ to P_1, where $(x[i][j]', y[i][j]') \leftarrow (x[i][\mathrm{g_i}(\mathrm{f}(j))], y[i][\mathrm{g_i}(\mathrm{f}(j))]), i \in [M]$, $j \in [L]$, $\mathrm{g_i}$ is a random permutation of order L, and any two $\mathrm{g_i}$ are identical with probability $1/M$. f is a random permutation of order L.

5 Private Comparison With Batching

In this section, we introduce the core concepts behind our PrivCMP, and formulate the thermometer encoding and dichotomy method of folklore. Then, we propose three PrivCMP algorithms: TECMP, CDCMP, and RDCMP.

5.1 PrivCMP overview

Step 1. We design PrivCMP by utilizing Thermometer Encoding (TE) [9] and LHE [22]. This approach is inspired by XCMP [28], where involves three polynomial products, X^a, X^{-b}, and $T = 1 + X + \cdots + X^{N-1}$, yielding $(X^{a-b} + X^{a-b+1} + \cdots + X^{a-b+N-1}) \mod (X^N - 1)$. The 0-th coefficient is then output as $\{-1, 1\}$. The polynomial product with X^a reflects a similar rotation operation in LHE, which allows us to implement it using LHE by One-Hot encoding the value of a into the vector $\boldsymbol{\theta}$ and applying rotation instead of the polynomial product. Most importantly, TE enables the simultaneous output of $\mathrm{I}(a > b)$ and $\mathrm{I}(a = b)$.

$$\boldsymbol{\theta} = [\overbrace{1, 1, \dots, 1}^{a+1}, 0, 0, \dots, 0] \in \{0,1\}^{2^n}, a \in [2^n] \tag{7}$$

$$\mathrm{I}(a > b) = \mathrm{Rotate}_b(\boldsymbol{\theta}), b \in [2^n] \tag{8}$$

$$\mathrm{I}(a = b) = \mathrm{I}(a > b - 1) - \mathrm{I}(a > b) \tag{9}$$

$$= \begin{cases} [1, 0, 0, \dots, 0] - \mathrm{Rotate}_b(\boldsymbol{\theta}), b \in \{0\} \\ \mathrm{Rotate}_{b-1}(\boldsymbol{a}) - \mathrm{Rotate}_b(\boldsymbol{\theta}), otherwise \end{cases} \tag{10}$$

Step 2. TE alone is insufficient to complete our PrivCMP, as the polynomial degree N limits the comparison to $\log_2 N$ precision. The next step is to leverage the features of LHE, where ciphertexts can be manipulated in subsequent operations. We observe that the recursive method in the folklore algorithm achieves linear precision growth with multiplicative depth. Therefore, we have introduced a new dichotomy method that achieves logarithmic precision growth with respect to multiplicative depth, making it more suitable for scenarios with limited multiplicative depth. The dichotomy method show below, where $a = a_0 + a_1 \cdot 2^{\frac{n}{2}}, b = b_0 + b_1 \cdot 2^{\frac{n}{2}}, a_i, b_i \in [2^{\frac{n}{2}}]$.

$$\mathrm{I}(b > a) = \mathrm{I}(a_1 > b_1) + \mathrm{I}(a_1 = b_1) \cdot \mathrm{I}(a_0 > b_0) \tag{11}$$

We then apply the same method to compute $\mathrm{I}(a_1 > b_1)$, $\mathrm{I}(a_1 = b_1)$, and $\mathrm{I}(a_0 > b_0)$. By combining TE with 2^m-base encoding and the dichotomy method, we complete the TECMP. This approach achieves very high precision (up to 26,624 bits), which is sufficient for most practical applications.

Step 3. We degenerate the TE with 2^m-base encoding into binary expansion, exclusively using the dichotomy method to enhance performance. This approach leads to the design of RDCMP and CDCMP, enabling two ciphertext inputs and reducing communication costs. The key distinction between RDCMP and CDCMP lies in their batching strategies: RDCMP uses vertical encoding for multiple vectors, while CDCMP employs horizontal encoding for a single vector.

5.2 Thermometer Encoding

TE [9] is an extension of One-Hot encoding that records the sorting information of values in a vector, which we use for comparisons. During the comparison of two numbers, if $\mathrm{I}(a > b) = 1, a, b \in [2^n]$, then, it follows that $\mathrm{I}(a > x) = 1, x \in [b + 1]$. as shown in Algorithm 1.

$$[\theta_0, \theta_1, \ldots, \theta_{2^n-1}] \leftarrow \mathrm{TE}(a, n), \theta_i = \mathrm{I}(a > i) \tag{12}$$

Algorithm 1 Thermometer Encoding (TE)

Procedure : Encode(a, n)
$\boldsymbol{\theta} \leftarrow 0^{2^n}$;
for all $i \in [a + 1]$ **do**
    $\theta[i] = 1$;
end for
return $\boldsymbol{\theta}$.

Compared with XCMP [28], the polynomial product between X^a and other terms is equivalent to rotating a slots in TE. Consequently, the $\mathrm{I}(a > b)$ are shown in Algorithm 2.

Algorithm 2 Greater Than $\mathrm{I}(a > b)$

Pre-procedure :
$\boldsymbol{\theta} \leftarrow$ TE.Encode(a, n);
Procedure : GT$(\boldsymbol{\theta}, b, n)$
$\boldsymbol{\theta_{gt}} \leftarrow \mathrm{Rotate}_b(\boldsymbol{\theta})$;
return $\boldsymbol{\theta_{gt}}$.

After executing Algorithm 2, the 0th slot of the output contains information about $\mathrm{I}(a > b)$. By performing a right rotation, we can obtain $\mathrm{I}(a > b-1)$ in the 0th slot. Then, we can derive $\mathrm{I}(a = b)$ as $\mathrm{I}(a > b-1) - \mathrm{I}(a > b)$, as shown in Algorithm 3. Lemma 1 demonstrates the correctness of the greater than and equal to algorithms.

Algorithm 3 Equal To $\mathrm{I}(a = b)$

Pre-procedure :
$\boldsymbol{\theta} \leftarrow$ TE.Encode(a, n);
$\boldsymbol{\theta_{gt}} \leftarrow$ TE.GT$(\boldsymbol{\theta}, b, n)$;
Procedure : ET$(\boldsymbol{\theta_{gt}}, b, n)$
if $b > 0$ **then**
    $\boldsymbol{\theta'_{gt}} \leftarrow \mathrm{Rotate}_{-1}(\boldsymbol{\theta_{gt}})$;
else
    $\boldsymbol{\theta'_{gt}} \leftarrow [1, 0, 0, \ldots, 0]$;
end if
$\boldsymbol{\theta_{eq}} \leftarrow \boldsymbol{\theta'_{gt}} \ominus \boldsymbol{\theta_{gt}}$;
return $\boldsymbol{\theta_{eq}}$.

Lemma 1: Algorithm 2 and Algorithm 3 are correct for input a, b and output $\mathrm{I}(a > b), \mathrm{I}(a = b)$ respectively.
Proof the lemma 1:
Thermometer encoding a to

$$\boldsymbol{\theta} = [\mathrm{I}(a > 0), \mathrm{I}(a > 1), \ldots, \mathrm{I}(a > (2^n - 1))] \tag{13}$$

Rotate b slots and obtained

$$\boldsymbol{\theta_{gt}} = [\mathrm{I}(a > b), \ldots, \mathrm{I}(a > 2^n - 1), \mathrm{I}(a > 0), \ldots, \mathrm{I}(a > b - 1)] \tag{14}$$

Then the 0th slot of $\boldsymbol{\theta_{gt}}$ is $\mathrm{I}(a > b)$.
When $b > 0$, $\boldsymbol{\theta_{gt}}$ Rotate -1 slot and obtained

$$\boldsymbol{\theta'_{gt}} = [\mathrm{I}(a > b - 1), \ldots, \mathrm{I}(a > 2^n - 1), \mathrm{I}(a > 0), \ldots, \mathrm{I}(a > b - 2)] \tag{15}$$

When $b = 0$, the $\mathrm{I}(a > b - 1) = 1$, for all $a \in [2^n]$.

$$\boldsymbol{\theta'_{gt}} = [1, 0, 0, \ldots, 0] = [\mathrm{I}(a > b - 1), 0, 0, \ldots, 0] \tag{16}$$

So the 0th slot of $\boldsymbol{\theta'_{gt}}$ is $\mathrm{I}(a > b - 1)$. Due to

$$\mathrm{I}(a = b) = \mathrm{I}(a > b - 1) - \mathrm{I}(a > b) \tag{17}$$

The 0th slot of $\boldsymbol{\theta_{eq}} \leftarrow \boldsymbol{\theta'_{gt}} \ominus \boldsymbol{\theta_{gt}}$ is $\mathrm{I}(a = b)$. □
After executing Algorithms 2 and 3, both results are stored in the 0th slot of the output. However, unrelated information may remain in the other slots, so the final output requires cleaning, as shown in Algorithm 4.

Algorithm 4 Clear Up

1: **Procedure :** $\mathrm{CU}(\boldsymbol{\theta_{in}}, w)$
2: $W \leftarrow 0^{2^n}$;
3: **for all** $j \in \{0, w, 2w, 3w, \ldots, 2^{n-1} - w\}$ **do**
4: $\quad W[j] = 1$;
5: $\quad W[2^{n-1} + j] = 1$;
6: **end for**
7: $\boldsymbol{\theta_{out}} \leftarrow \boldsymbol{\theta_{in}} \otimes W$;
8: **return** $\boldsymbol{\theta_{out}}$.

In TE, the encoded vector expands exponentially relative to the input value size. For example, comparing two 8-bit numbers requires 256 slots to encode one of them. Given a polynomial with an upper limit on slots, such as 2^{14}, this means that TE supports only up to 13-bit precision. However, this limitation can be addressed in our algorithm based on LHE. The results of $\mathrm{I}(a > b)$ and $\mathrm{I}(a = b)$ are stored in the 0th slot, enabling further Hadamard addition and multiplication without requiring additional operations, thereby facilitating straightforward precision expansion. Next, we introduce the recursive and dichotomy methods from folklore and demonstrate how they are used to construct our PrivCMP.

5.3 A Dichotomy Method of Folklore Algorithm

Observe that the original folklore computations take as input $a = \sum_{i=0}^{n-1} 2^i a_i$, $b = \sum_{i=0}^{n-1} 2^i b_i$, then compute $g_i \leftarrow \mathrm{I}(a_i > b_i), e_i \leftarrow \mathrm{I}(a_i = b_i)$, the output is $\mathrm{I}(a > b) = \theta_{norm}$.

$$\theta_{norm} = \sum_{i=0}^{n-1} g_i \cdot \prod_{k=i+1}^{n-1} e_k \tag{18}$$

This can also be performed recursively (Algorithm 5).

$$\theta_{rec} = \theta_{n-1} \tag{19}$$
$$\theta_{k-1} = g_{k-1} + e_{k-1} \cdot \theta_{k-2} \tag{20}$$
$$\theta_0 = g_0 \tag{21}$$

And a new dichotomy way (Algorithm 6).

$$\theta_{dic} = g_{\{s+1:n-1\}} + e_{\{s+1:n-1\}} \cdot g_{\{0:s\}} \tag{22}$$
$$g_{\{i:j\}} = \mathrm{I}(a_{\{i:j\}} > b_{\{i:j\}}) \tag{23}$$
$$e_{\{i:j\}} = \mathrm{I}(a_{\{i:j\}} > b_{\{i:j\}}) \tag{24}$$
$$a_{\{i:j\}} = a_i + a_{i+1} \cdot 2^1 + \cdots + a_j \cdot 2^{j-i} \tag{25}$$
$$b_{\{i:j\}} = b_i + b_{i+1} \cdot 2^1 + \cdots + b_j \cdot 2^{j-i} \tag{26}$$

The recursive approach is natural, computing the final result bit by bit, while the normal approach unfolds the recursive method to reduce multiplicative depth. The dichotomy method, inspired by the multiply multiple ciphertexts in LHE, follows a tower-like structure. As shown in Table 3, our dichotomy method maintains the multiplicative depth while reducing the number of multiplications compared to the normal method, and it has a lower multiplicative depth than the recursive approach. Lemma 2 demonstrates the correctness of the dichotomy method in Algorithm 6.

Table 3. Three Methods of folklore

Method	Normal	Recursive	Dichotomy
Mul. depth	$\lceil log(n) \rceil$	$n-1$	$\lceil log(n) \rceil$
Mul. times	$n \cdot (n-1)/2$	$n-1$	$2 \cdot (n-1)$

Algorithm 5 Recursive Method

Procedure : RM($\boldsymbol{\theta_{gt}}, \boldsymbol{\theta_{eq}}, n$)
$\theta_{rec} = \theta_{gt}[0]$
for all $i \in [n] \backslash \{0\}$ **do**
  $\theta_{rec} \leftarrow \theta_{rec} \odot \theta_{eq}[i] \oplus \theta_{gt}[i]$;
end for
return θ_{rec}.

Algorithm 6 Dichotomy Method, n is power of 2

Procedure : DM($\boldsymbol{\theta_{gt}}, \boldsymbol{\theta_{eq}}, n$)
for all $i \in [\log_2 n]$ **do**
  $t = 1 \ll i$;
  $w = 1 \ll (i+1)$;
  for all $j \in \{0, w, 2w, \ldots, n-w\}$ **do**
    $\theta_{gt}[j] \leftarrow \theta_{gt}[j] \odot \theta_{eq}[j+t] \oplus \theta_{gt}[j+t]$;
    $\theta_{eq}[j] \leftarrow \theta_{eq}[j] \odot \theta_{eq}[j+t]$;
  end for
end for
$\theta_{dic} = \theta_{gt}[0]$;
return θ_{dic}.

Lemma 2: Algorithm 6 is correct for input $\mathrm{I}(a_i = b_i), \mathrm{I}(a_i > b_i)$ and output $\mathrm{I}(a > b)$.
Proof the lemma 2: Set

$$\boldsymbol{\theta_{gt}} = \{g_0, g_1, \ldots, g_{n-1}\}, g_i \leftarrow \mathrm{I}(a_i > b_i) \tag{27}$$
$$\boldsymbol{\theta_{eq}} = \{e_0, e_1, \ldots, e_{n-1}\}, e_i \leftarrow \mathrm{I}(a_i = b_i) \tag{28}$$

Let $0 \le i \le k < j \le n$, and

$$g_{\{i:j\}} = \mathrm{I}(a_{\{i:j\}} > b_{\{i:j\}}) \tag{29}$$
$$e_{\{i:j\}} = \mathrm{I}(a_{\{i:j\}} = b_{\{i:j\}}) \tag{30}$$

Notes that

$$g_{\{i:j\}} = g_{\{k+1:j\}} + e_{\{k+1:j\}} \cdot g_{\{i:k\}} \tag{31}$$
$$e_{\{i:j\}} = e_{\{k+1:j\}} \cdot e_{\{i:k\}} \tag{32}$$

So the lemma 2 is hold. □

An example with 8-bit values is shown in the Figure 3. At last, the $\{g_i, e_i\}$ are computed by $\{a_i \cdot (1-b_i), 1-(a_i-b_i)^2\}$ in original folklore. We can compute $b'_i = 1 - b_i$, and then update $\{g_i, e_i\} \leftarrow \{a_i \cdot b'_i, b'_i - 2 \cdot g_i + a_i\}$. This requires at most one product and one multiplicative depth.

5.4 Thermometer Encoding Comparison (TECMP)

In the folklore algorithm, once the $g_i \leftarrow \mathrm{I}(a_i > b_i), e_i \leftarrow \mathrm{I}(a_i = b_i)$ are computed, the $\theta = \mathrm{I}(a > b)$ can be derived using the normal, recursive and dichotomy methods. By replacing the binary expansion to m-bit representation,

$\overrightarrow{\theta_{gt}}$	g_0	g_1	g_2	g_3	g_4	g_5	g_6	g_7
$\overrightarrow{\theta_{eq}}$	e_0	e_1	e_2	e_3	e_4	e_5	e_6	e_7
$g_{2i} = g_{2i+1} + e_{2i+1} \cdot g_{2i}$	g_0		g_2		g_4		g_6	
$e_{2i} = e_{2i+1} \cdot e_{2i}$	e_0		e_2		e_4		e_6	
$g_{4i} = g_{4i+2} + e_{4i} \cdot g_{4i}$	g_0				g_4			
$e_{4i} = e_{4i+2} \cdot e_{4i}$	e_0				e_4			
$g_{8i} = g_{8i+4} + e_{8i} \cdot g_{8i}$	g_0							

Fig. 3. Dichotomy example of 8-bit number

such as $g_{\{i:i+m\}} \leftarrow \mathrm{I}(a_{\{i:i+m\}} > b_{\{i:i+m\}}), e_{\{i:i+m\}} \leftarrow \mathrm{I}(a_{\{i:i+m\}} = b_{\{i:i+m\}})$, the process remains work effective. Specifically, the numbers a are expanded by m-bit, encoding each $a_{\{i:i+m\}}$ with TE.

$$a = \sum_{i=0}^{l-1} 2^{m \cdot i} \cdot a_{\{i:i+m\}}, a_{\{i:i+m\}} \in [2^m] \tag{33}$$

Using Algorithm 2, 3 to compute each $\{g_{\{i:i+m\}}, e_{\{i:i+m\}}\}$, we can then compute $\mathrm{I}(a > b)$, as shown in Algorithm 7 with the dichotomy method.

Algorithm 7 Thermometer Encoding Comparison $\mathrm{I}(a > b)$, $a, b \in [2^{m \cdot l}]$, l is power of 2.

Procedure : Encode(a, b)
for all $i \in [l]$ **do**
  $a_i \leftarrow (a \gg i \cdot m) \& (2^m - 1)$;
  $b_i \leftarrow (b \gg i \cdot m) \& (2^m - 1)$;
  $\boldsymbol{\theta_i} \leftarrow \mathrm{TE.Encode}(a_i, m)$;
end for
Procedure : $\mathrm{GT}(\{\boldsymbol{\theta_0}, \ldots, \boldsymbol{\theta_{l-1}}\}, \{b_0, \ldots, b_{l-1}\}, l)$
for all $i \in [l]$ **do**
  $\boldsymbol{\theta_{gt}[i]} \leftarrow \mathrm{TE.GT}(\boldsymbol{\theta_i}, b_i, l)$;
  $\boldsymbol{\theta_{eq}[i]} \leftarrow \mathrm{TE.ET}(\boldsymbol{\theta_{gt}[i]}, b_i, l)$;
end for
$\boldsymbol{\theta_{TECMP}} = \mathrm{DM}(\boldsymbol{\theta_{gt}}, \boldsymbol{\theta_{eq}}, l)$;
return $\mathrm{CU}(\boldsymbol{\theta_{TECMP}}, 2^m)$.

In Algorithm 7, we only use 2^m slots in an N slots vector to encode a number. The rest can while the remaining slots can be utilized for other numbers to enable batching comparisons, as illustrated in Figure 4 (a). Our TECMP effectively combines the strengths of folklore and TE. It minimizes the number of product to reduce multiplicative depth and mitigates exponential slot expansion. This approach strikes a balance between the number of multiplications, multiplicative depth, and batch size, resulting in a relatively low amortized computing cost. However, TECMP supports only plaintext inputs and many-to-one comparisons, which may limit its broader applicability. To overcome this limitation, we propose RDCMP and CDCMP to further enhance its versatility.

5.5 Row Dichotomy Comparison (RDCMP)

To address the plaintext input limitation of TECMP and enable the PrivCMP to support two ciphertext inputs. We propose a solution that focuses solely on binary expansion, computing the $\{g_i, e_i\}_{i \in [n]}$ without the need for rotation, thus eliminating the plaintext input.

The dichotomy method used in RDCMP can effectively reduce the multiplicative depth. Since the multiplicative depth is $\lceil \log_2 n \rceil$, we can support high-precision comparison, and the number of multiplications are $2n - 2$, which is significantly less than the $\frac{n \cdot (n-1)}{2}$ in normal method. It's important to note that we can view TE as a mechanism for balancing the space and time. If we don't strictly require the arbitrary input type, TE and binary expansion can be intermixed in any segment, as long as we compute $\{g_i, e_i\}$ for each segment.

5.6 Column Dichotomy Comparison (CDCMP)

In packaging strategy show in Figure 4 (b), RDCMP has n vectors when considering n-bit precision. We may desire fewer vectors, therefore, we pack $\{a_0, a_1, \ldots, a_{n-1}\}$ into a single vector as shown in Figure 4 (c).

Compared with RDCMP and CDCMP (Algorithm 9), the main difference is how the slots of the binary expansion are arranged. The former uses n vectors to separately store the n-bit value, while the later stores in a single vector, resulting in fewer multiplications and smaller batch size, but involves some rotations. Finally,

Algorithm 8 Row Dichotomy Comparison $\mathrm{I}(a > b), a, b \in [2^n]$, n is power of 2.

Procedure : Encode(a, b)
for all $i \in [n]$ **do**
  $a_i \leftarrow (a \gg i)\&1$;
  $b'_i \leftarrow 1 - (b \gg i)\&1$;
end for
Procedure : GT$(\{a_0, \ldots, a_{n-1}\}, \{b'_0, \ldots, b'_{n-1}\}, n)$
for all $i \in [n]$ **do**
  $\theta_{gt}[i] \leftarrow a_i \odot b'_i$;
  $\theta_{eq}[i] \leftarrow a_i \oplus b'_i \ominus \theta_{gt}[i] \ominus \theta_{gt}[i]$;
end for
$\boldsymbol{\theta_{RDCMP}} = \mathrm{DM}(\boldsymbol{\theta_{gt}}, \boldsymbol{\theta_{eq}}, n)$;
return $\boldsymbol{\theta_{RDCMP}}$.

Algorithm 9 Column Dichotomy Comparison $\mathrm{I}(a > b), a, b \in [2^n]$, n is power of 2.

Procedure : Encode(a, b)
for all $i \in [n]$ **do**
  $a_i \leftarrow (a \gg i)\&1$;
  $b'_i \leftarrow 1 - (b \gg i)\&1$;
end for
Procedure : GT$(\{a_0, \ldots, a_{n-1}\}, \{b'_0, \ldots, b'_{n-1}\}, n)$
$\boldsymbol{\theta_a} = [a_0, \ldots, a_{n-1}]$;
$\boldsymbol{\theta'_b} = [b'_0, \ldots, b'_{n-1}]$;
$\boldsymbol{\theta_{gt}} \leftarrow \boldsymbol{\theta_a} \odot \boldsymbol{\theta'_b}$;
$\boldsymbol{\theta_{eq}} \leftarrow \boldsymbol{\theta_a} \oplus \boldsymbol{\theta'_b} \ominus \boldsymbol{\theta_{gt}} \ominus \boldsymbol{\theta_{gt}}$;
for all $i \in [\log_2 n]$ **do**
  $t = 1 \ll i$;
  $\boldsymbol{\theta'_{gt}} \leftarrow \mathrm{Rotate}_t \boldsymbol{\theta_{gt}}$;
  $\boldsymbol{\theta'_{eq}} \leftarrow \mathrm{Rotate}_t \boldsymbol{\theta_{eq}}$;
  $\boldsymbol{\theta_{eq}} \leftarrow \boldsymbol{\theta_{eq}} \odot \boldsymbol{\theta'_{eq}}$;
  $\boldsymbol{\theta_{gt}} \leftarrow \boldsymbol{\theta_{gt}} \odot \boldsymbol{\theta'_{eq}} \oplus \boldsymbol{\theta'_{gt}}$;
end for
$\boldsymbol{\theta_{CDCMP}} = \mathrm{CU}(\boldsymbol{\theta_{gt}}, n)$;
return $\boldsymbol{\theta_{CDCMP}}$.

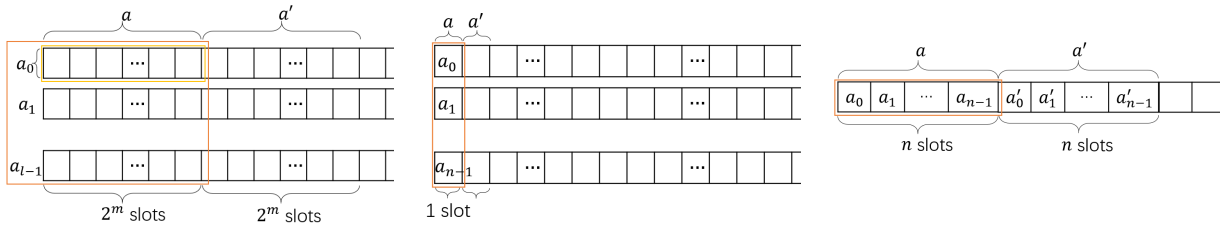
we define PrivCMP: P_1 inputs the vector $\boldsymbol{a}$ and P_2 inputs the vector $\boldsymbol{b}$, then output the ciphertexts $\{\mathrm{I}(a[i] > b[i]), i \in [\mathrm{len}(\boldsymbol{a})]\}$ that P_1 can decrypt. Our TECMP, RDCMP, and CDCMP are collectively referred to as PrivCMP.$\{\mathrm{Encode}, \mathrm{GT}\}$, and the interactive model is show in Algorithm 10, the theoretically properties show in Table 4.

Table 4. PrivCMP, the polynomial degree is N

Symbol	TECMP	RDCMP	CDCMP
Bit precision	$l \cdot m$	n	n
Mul. depth	$\lceil \log_2 l \rceil$	$\lceil \log_2 n \rceil$	$\lceil \log_2 n \rceil$
Vector numbers	l	n	1
Mul. numbers	$2 \cdot (l-1)$	$2 \cdot (n-1)$	$1 + 2 \cdot \log_2 n$
Rotate numbers	$2 \cdot m$	0	$1 + 2 \cdot \log_2 n$
Input type	plain-cipher	plain-cipher	plain-cipher
		cipher-cipher	cipher-cipher
	many to one	many to many	many to many
Batch size	$2 \cdot (N/2^m)$	N	N/n

Algorithm 10 Private Comparison $\{\mathrm{I}(a[0] > b[0]), \cdots\}$

Input : P_1 has $\boldsymbol{a}$, P_2 has $\boldsymbol{b}$.
The protocol :
P_1 run LHE.KeyGen$(1^\lambda, 1^L)$ to obtain (pk, sk, evk). P_1 send evk to P_2.
P_1 run PrivCmp.Encode$(a[i], \perp)$ in each element of $\boldsymbol{a}$ and obtain $a[i]'$, set $\boldsymbol{a'} = [a[0]'||...||a[\mathrm{len}(\boldsymbol{a})]']$, then get $c_{\boldsymbol{a'}}$ from LHE.Enc$(pk, \boldsymbol{a'})$. P_1 send $c_{\boldsymbol{a'}}$ to P_2.
P_2 run PrivCmp.Encode$(\perp, b[i])$ in each element of $\boldsymbol{b}$ and obtain $b[i]'$, set $\boldsymbol{b'} = [b[0]'||...||b[\mathrm{len}(\boldsymbol{b})]']$, then run PrivCmp.GT$(c_{\boldsymbol{a'}}, \boldsymbol{b'}, evk)$ to obtain the ciphertext $c_{\boldsymbol{\theta'}}$. P_2 send $c_{\boldsymbol{\theta'}}$ to P_1.
P_1 run LHE.Dec$(sk, c_{\boldsymbol{\theta'}})$ to obtain the output $\boldsymbol{\theta'}$.



(a) TECMP encoding (b) RDCMP encoding (c) CDCMP encoding

Fig. 4. Encoding visualization of PrivCMP

Theorem 1: The Algorithm 10 is correct for input $\boldsymbol{a}, \boldsymbol{b}$ and outputs $\{\mathrm{I}(a[i] > b[i]), i \in [\mathrm{len}(\boldsymbol{a})]\}$.

Proof the theorem 1: By Lemma 1, and Lemma 2, the Algorithm 7, 8, 9 is correct for input a, b and outputs $\mathrm{I}(a > b)$. Sequentially arrange each PrivCMP.Encode$(a[i], b[i])$, where $i \in [\mathrm{len}(\boldsymbol{a})]$. After performing PrivCMP.GT and decryption, then output $\{\mathrm{I}(a[i] > b[i]), i \in [\mathrm{len}(\boldsymbol{a})]\}$, so the Theorem 1 is holds. □

6 Batch Private Decision Tree Evaluation

This section presents our BPDTE schemes and introduces a new CRR algorithm to extend the SumPath.

6.1 BPDTE overview

Step 1. After designing these PrivCMP algorithm, we leverage them to develop the BPDTE. Our schemes incorporate the adapted SumPath (ASM) proposed in [15]. Step 2. We design a CRR algorithm to preserve privacy by randomly permuting the indices and values, thereby obfuscating the output positions and values. We then extend the SumPath method introduced in [2] to support batch operation. Step 3. We provide a comprehensive security analysis of our PrivCMP, BPDTE , and CRR within a simulation-based framework, along with a benchmark comparison against state-of-the-art methods in the following section.

6.2 BPDTE: PrivCMP with SumPath

It is assumed that the public key and evaluation key have been exchanged between the client and the server. We integrate PrivCMP and SumPath in BPDTE, which primarily consists of two parts: For each internal node, privately compare the client attribute with the node's threshold to obtain the node's state value; Running SumPath to output the classification results.

In [26], Cong et al. propose a ASM, using a function map the $\{0, 1, \ldots, u\}$ to $\{0, 1\}$,

$$\mathrm{f}(x) = \frac{1}{(u-1)!} \cdot \prod_{i=1}^{u-1} (i - x) \tag{34}$$

then use a private information retrieval technique to output a leaf node. We present it in Algorithm 11 and the our BPDTE are show in Algorithm 12.

Algorithm 11 Adapted SumPath

Procedure : ASM$(\boldsymbol{c}, \boldsymbol{v}, d, L)$
for all $d \in D$ **do**
  $d.left \leftarrow c[d]$;
  $d.right \leftarrow 1 - c[d]$;
end for
for all $l \in L$ **do**
  $s[l] =$ Sum the d value from the root to leaf l;
  $u =$ The depth from the root to leaf l;
  $s[l] = 1/(u-1)! \cdot \prod_{i=1}^{u-1}(i - s[l])$;
end for
$z \leftarrow 0$;
for all $l \in L$ **do**
  $z = z + s[l] \cdot v[l]$;
end for
return z.

Theorem 2: Algorithm 12 is correct for inputs attribute vector $\boldsymbol{x} = [\boldsymbol{x}[\boldsymbol{0}], \ldots, \boldsymbol{x}[\boldsymbol{M-1}]]^T$ and decision tree $\mathcal{M}$, and outputs $\{\mathcal{M}(\boldsymbol{x}[\boldsymbol{0}]), \ldots, \mathcal{M}(\boldsymbol{x}[\boldsymbol{M-1}])\}$.

Proof the theorem 2: If Theorem 1 holds, Algorithm 12 is correct for completing the node comparison. Since the ASM in Algorithm 11 is a function that maps $\{0, 1, \ldots, u\}$ to $\{0, 1\}$ and only one leaf being 1 in the output, the correctness of BPDTE under the ASM holds. □

Algorithm 12 Batch Private Decision Tree Evaluation

Procedure : Encode($\boldsymbol{x}, \mathcal{M}$)
$\boldsymbol{x} = [\boldsymbol{x[0]}, \ldots, \boldsymbol{x[L-1]}]$;
$\boldsymbol{x[j]} = [x[0][j], \ldots, x[M-1][j]]^T$;
for all $j \in [L]$ **do**
  $\boldsymbol{x[j]'} = \perp$;
  for all $i \in [M]$ **do**
    $\boldsymbol{\theta} \leftarrow$ PrivCmp.Encode($x[i][j], \perp$);
    $\boldsymbol{x[j]'} = \boldsymbol{x[j]'} || \boldsymbol{\theta}$;
  end for
end for
$\boldsymbol{x'} \leftarrow [\boldsymbol{x[0]'}, \ldots, \boldsymbol{x[L-1]'}]$;
$\{L, \boldsymbol{t}, \boldsymbol{a}, \boldsymbol{v}, D\} \leftarrow \mathcal{M}$;
$\boldsymbol{t} = [t[0], \ldots, t[D-1]]$;
for all $d \in D$ **do**
  $\boldsymbol{\theta} \leftarrow$ PrivCmp.Encode($\perp, t[d]$);
  $\boldsymbol{t[d]'} = \boldsymbol{\theta} || \ldots || \boldsymbol{\theta}$;
end for
$\boldsymbol{t'} = [\boldsymbol{t[0]'}, \ldots, \boldsymbol{t[D-1]'}]$;
Procedure : Eval($\boldsymbol{x'}, \boldsymbol{t'}, \mathcal{M}$)
for all $d \in D$ **do**
  $c[d] \leftarrow$ PrivCmp.GT($\boldsymbol{x[a[d]]'}, \boldsymbol{t[d]'}$);
end for
$z \leftarrow$ ASM($\{c[0], \ldots, c[D-1]\}, \boldsymbol{v}, d, L$);
return z

Algorithm 13 Batch Private Decision Tree Evaluation

Input : P_1 has $\boldsymbol{x} = [\boldsymbol{x[0]}, \ldots, \boldsymbol{x[L-1]}]$, P_2 has $\mathcal{M}$.
The protocol :
P_2 run $\mathcal{M}$.Params(1^λ) to obtain L_1. P_2 send L_1 to P_1.
P_1 run PrivCmp.Params(1^λ) to obtain L_2, and run LHE.KeyGen($1^\lambda, 1^{L_1+L_2}$) to obtain (pk, sk, evk). P_1 send evk to P_2.
P_1 run BPDTE.Encode($\boldsymbol{x}, \perp$) to obtain $\boldsymbol{x'}$, then get each $c_{\boldsymbol{x[i]'}}$ from LHE.Enc($pk, \boldsymbol{x[i]'}$). P_1 send $c_{\boldsymbol{x'}} \leftarrow \{c_{\boldsymbol{x[0]'}}, \ldots, c_{\boldsymbol{x[L-1]'}}\}$ to P_2.
P_2 run BPDTE.Encode($\perp, \mathcal{M}$) to obtain $\boldsymbol{t'}$, then run BPDTE.Eval($c_{\boldsymbol{x'}}, \boldsymbol{t'}, \mathcal{M}, evk$) to obtain the ciphertext $c_{\boldsymbol{\theta'}}$. P_2 send $c_{\boldsymbol{\theta'}}$ to P_1.
P_1 run LHE.Dec($sk, c_{\boldsymbol{\theta'}}$) to obtain the output $\boldsymbol{\theta'}$.

6.3 Extended SumPath

In the SumPath of Level Up [2], two random vectors are used to conceal the classification result. However, we identified that the final classification result depends on the position of zeros, which violates the purpose of BPDTE. The goal is for the client to learn the final classification result without revealing any additional information. To address this problem, we design a CRR in Algorithm 14 that obfuscates the position of the classification results. The improved version is referred to as the extended SumPath (ESM) in Algorithm 15.

In Algorithm 14, we expect that the probability of a semi-honest client determining that two classification results are at the same position does not exceed $1/M$, where M represents the number of rows in the client's data (i.e. the batch sizes). Specifically, since these $2 \cdot L$ ciphertexts in $\{\boldsymbol{x}, \boldsymbol{y}\}$ record the classification results of M rows of data, we first perform a random permutation $\mathrm{P_L}$ to obscure the original positions. Then, we randomly split and merge half of each rows of $\{\boldsymbol{x}, \boldsymbol{y}\}$, repeating this process $\log_2 M$ times.

Lemma 6: Algorithm 14 is correct for inputs $(\boldsymbol{x}, \boldsymbol{y})$ and outputs $(x[i][\mathrm{g_i}(\mathrm{f}(j))], y[i][\mathrm{g_i}(\mathrm{f}(j))]), i \in [M], j \in [L]$, where $\mathrm{g_i}$ is a random permutation of order L, and any two $\mathrm{g_i}$ are identical with probability $1/M$, f is a random permutation.

Proof the lemma 6: First, we need explain the input the $(\boldsymbol{x}, \boldsymbol{y})$, where

$$\boldsymbol{x} = [\boldsymbol{x[0]}, \ldots, \boldsymbol{x[L-1]}, \boldsymbol{x[j]} \in Z_p^M] \tag{35}$$

$$\boldsymbol{y} = [\boldsymbol{y[0]}, \ldots, \boldsymbol{y[L-1]}, \boldsymbol{y[j]} \in Z_p^M] \tag{36}$$

$$\boldsymbol{x[j]} = \begin{bmatrix} x[0][j] \\ \vdots \\ x[M-1][j] \end{bmatrix}, \boldsymbol{y[j]} = \begin{bmatrix} y[0][j] \\ \vdots \\ y[M-1][j] \end{bmatrix} \tag{37}$$

Each row of $\boldsymbol{x}$ and $\boldsymbol{y}$ stores one classification result, with a total of M classification results. When inputting $(\boldsymbol{x}, \boldsymbol{y})$, first apply a random permutation $\mathrm{P_L}$.

$$\boldsymbol{x} = [\boldsymbol{x[0]}, \ldots, \boldsymbol{x[L-1]}] \rightarrow [\boldsymbol{x[\mathbf{P_L}(0)]}, \ldots, \boldsymbol{x[\mathbf{P_L}(L-1)]}] \tag{38}$$

$$\boldsymbol{y} = [\boldsymbol{y[0]}, \ldots, \boldsymbol{y[L-1]}] \rightarrow [\boldsymbol{y[\mathbf{P_L}(0)]}, \ldots, \boldsymbol{y[\mathbf{P_L}(L-1)]}] \tag{39}$$

Algorithm 14 Clear Rows Relation

Procedure : $\mathrm{CRR}(\boldsymbol{x}, \boldsymbol{y}, M, L)$
$\mathrm{P_L}$ = Random permutation in $[L]$;
for all $l \in [L]$ **do**
  $x'[l] = x[\mathrm{P_L}(l)]$;
  $y'[l] = y[\mathrm{P_L}(l)]$;
end for
for all $i \in [\log_2 M]$ **do**
  $\mathrm{P_M}$ = Random permutation in $[M]$;
  $W_0, W_1 \leftarrow 0^M$;
  for all $j \in [M/2]$ **do**
    $W_0[\mathrm{P_M}(j)] = 1$;
    $W_1[\mathrm{P_M}(M/2 + j)] = 1$;
  end for
  P'_L = Random permutation in $[L]$;
  for all $l \in [L]$ **do**
    $x'_0[l] = x'[l] \odot W_0$;
    $x'_1[l] = x'[l] \odot W_1$;
    $y'_0[l] = y'[l] \odot W_0$;
    $y'_1[l] = y'[l] \odot W_1$;
  end for
  for all $l \in [L]$ **do**
    $x'[l] = x'_0[\mathrm{P'_L}(l)] \oplus x'_1[l]$;
    $y'[l] = y'_0[\mathrm{P'_L}(l)] \oplus y'_1[l]$;
  end for
end for
return $\{\boldsymbol{x}', \boldsymbol{y}'\}$.

So, the original relationship between the indices of $\boldsymbol{x}$ and the leaf positions is obscured.

Secondly, we need to break the relationship between any two different rows. Use the random permutation $\mathrm{P_M}$ to randomly separate each $\boldsymbol{x}[j]$ and $\boldsymbol{y}[j]$ into two parts.

$$\boldsymbol{x_0}[j] = \begin{bmatrix} x[\mathrm{P_M}(0)][\mathrm{P_L}(j)] \\ \vdots \\ x[\mathrm{P_M}(M/2-1)][\mathrm{P_L}(j)] \end{bmatrix} \tag{40}$$

$$\boldsymbol{x_1}[j] = \begin{bmatrix} x[\mathrm{P_M}(M/2)][\mathrm{P_L}(j)] \\ \vdots \\ x[\mathrm{P_M}(M-1)][\mathrm{P_L}(j)] \end{bmatrix} \tag{41}$$

$$\boldsymbol{y_0}[j] = \begin{bmatrix} y[\mathrm{P_M}(0)][\mathrm{P_L}(j)] \\ \vdots \\ y[\mathrm{P_M}(M/2-1)][\mathrm{P_L}(j)] \end{bmatrix} \tag{42}$$

$$\boldsymbol{y_1}[j] = \begin{bmatrix} y[\mathrm{P_M}(M/2)][\mathrm{P_L}(j)] \\ \vdots \\ y[\mathrm{P_M}(M-1)][\mathrm{P_L}(j)] \end{bmatrix} \tag{43}$$

Then, use $\mathrm{P'_L}$ to permute $\boldsymbol{x_0}[j]$ and $\boldsymbol{y_0}[j]$ at the same time.

$$[\boldsymbol{x_0}[\boldsymbol{0}], \ldots, \boldsymbol{x_0}[\boldsymbol{L-1}]] \rightarrow [\boldsymbol{x_0}[\mathbf{P'_L}(\boldsymbol{0})], \ldots, \boldsymbol{x_0}[\mathbf{P'_L}(\boldsymbol{L-1})]] \tag{44}$$

$$[\boldsymbol{y_0}[\boldsymbol{0}], \ldots, \boldsymbol{y_0}[\boldsymbol{L-1}]] \rightarrow [\boldsymbol{y_0}[\mathbf{P'_L}(\boldsymbol{0})], \ldots, \boldsymbol{y_0}[\mathbf{P'_L}(\boldsymbol{L-1})]] \tag{45}$$

Half of the rows are obscured, and recursive calls are made $\log_2 M$ times to obtain the result.

$$x[i][j] \rightarrow x[i][\mathrm{P_L^i}(\mathrm{P_L}(j))] \tag{46}$$

$$y[i][j] \rightarrow y[i][\mathrm{P_L^i}(\mathrm{P_L}(j))] \tag{47}$$

where $\mathrm{P_L^i} \leftarrow_r \{\mathrm{P_L^1}, \ldots, \mathrm{P_L^M}\}$. Any two $\mathrm{P_L^i}$ are the same with a probability of $1/M = (1/2)^{\log_2 M}$. □

The correctness of BPDTE under ESM also holds if Lemma 6 holds, because for each row of data, only one leaf is 0 in the output of ESM. In general, the ASM requires $d-1$ instances of multiplications for each leaf node and a minimum multiplicative depth of $\lceil \log_2(d-1) \rceil$, but it benefits from small and constant communication costs. On the other hand, ESM doesn't require multiplications and minimum multiplicative depth, but it involves substantial circular computations and large outputs, which increase the noise budgets and lead to higher communication costs.

Algorithm 15 Extended SumPath

Procedure : ESM($\boldsymbol{c}, \boldsymbol{v}, M.L$)
for all $d \in D$ **do**
  $d.left \leftarrow c[d]$;
  $d.right \leftarrow 1 - c[d]$;
end for
for all $l \in [L]$ **do**
  $s[l] =$ Sum the d value from the root to leaf l;
  $r_x, r_y \leftarrow_r Z_p^N$;
  $x[l] \leftarrow r_x \odot s[l]$;
  $y[l] \leftarrow r_y \odot s[l] + v[l]$;
end for
$\{\boldsymbol{x}', \boldsymbol{y}'\} \leftarrow$ CRR($\boldsymbol{x}, \boldsymbol{y}, M, L$)
return $\{\boldsymbol{x}', \boldsymbol{y}'\}$.

7 Security Analysis

7.1 Security overview

In PrivCMP, the goal is to provide a comparison output between two input numbers while ensuring that no information is leaked to an honest eavesdropper. All messages are LHE ciphertexts, and the semantic security of the LHE guarantees the confidentiality of ours PrivCMP. In PDTE, based on PrivCMP, neither the honest server nor any eavesdropper possesses sensitive information, as they lack decryption capabilities, and all messages they receive are ciphertexts. The client will only receive the classification result and nothing else. In CRR, for an honest client, each separation and subsequent merging has a probability of $\frac{1}{2}$ of confusing the positions. Therefore, in a circular computation performed t times, the probability of distinguishing the relationship between the classification results and the leaf nodes is $\frac{1}{2^t}$.

7.2 Security proof

Theorem 3: If the LHE is semantically secure, then Algorithm 10 securely implements the ideal PrivCMP functionality in the semi-honest model.

Proof the theorem 3: The correctness is guaranteed by Theorem 1. Now, we need to construct the simulator $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{cmp}}$ for P_1's view and $\mathrm{Sim}_{\mathrm{P}_2}^{\mathrm{cmp}}$ for P_2's view.

First, note that P_1's input is $\boldsymbol{a} = [a[0], \ldots, a[k-1]]$ and its output is $\boldsymbol{\theta'} = [\theta[0]', \ldots, \theta[k-1]']$. Therefore, for each $i \in [k]$, P_1 immediately knows that $b[i] \in \{0, \ldots, a[i]-1\}$ if $\theta[i]' = 1$, or $b[i] \in \{a[i], \ldots, 2^n - 1\}$ if $\theta[i]' = 0$, but doesn't know the exact value of $b[i]$. The $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{cmp}}$ has the public parameters λ and L, along with P_1's input $\boldsymbol{a}$ and output $\boldsymbol{\theta'}$. Then:

1. $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{cmp}}$ runs LHE.KeyGen($1^\lambda, 1^L$) to obtain pk, sk.
2. $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{cmp}}$ computes $\sigma[i]' \leftarrow$ PrivCmp.Encode($\sigma[i]$) and obtain $\boldsymbol{\sigma'} = [\sigma[0]' || \ldots || \sigma[k-1]']$, where $\sigma[i] = \theta[i]'$.
3. Computes $c_{\boldsymbol{\sigma'}} \leftarrow$ LHE.Enc($pk, \boldsymbol{\sigma'}$).
4. $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{cmp}}$ outputs $(a; c_{\boldsymbol{\sigma'}})$.

We note that $\mathrm{View}_{\mathrm{P}_1}^{\Pi_{\mathrm{cmp}}}(\boldsymbol{a}, \boldsymbol{b}) = (\boldsymbol{a}; c_{\boldsymbol{\theta'}})$. Since $c_{\boldsymbol{\sigma'}}$ and $c_{\boldsymbol{\theta'}}$ are indistinguishable due to the semantically secure of LHE, and even if P_1 runs LHE.Dec($sk, c_{\boldsymbol{\theta'}}$) and LHE.Dec($sk, c_{\boldsymbol{\sigma'}}$) to obtain $\boldsymbol{\theta'}$ and $\boldsymbol{\sigma'}$, they remain indistinguishable because they are the same value. Thus, there exists a simulator such that $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{cmp}}(\boldsymbol{a}, \boldsymbol{\theta'}) \equiv_c \mathrm{View}_{\mathrm{P}_1}^{\Pi_{\mathrm{cmp}}}(\boldsymbol{a}, \boldsymbol{b})$.

Secondly, we need to construct a simulator $\mathrm{Sim}_{\mathrm{P}_2}^{\mathrm{cmp}}$ that has the public parameters λ and L, along with P_2's input $\boldsymbol{b}$, since P_2 has no output. Then:

1. $\mathrm{Sim}_{\mathrm{P}_2}^{\mathrm{cmp}}$ runs LHE.KeyGen($1^\lambda, 1^L$) to obtain pk.
2. $\mathrm{Sim}_{\mathrm{P}_2}^{\mathrm{cmp}}$ computes $r[i]' \leftarrow$ PrivCmp.Encode($r[i]$) and obtain $\boldsymbol{r'} = [r[0]' || \ldots || r[k-1]']$, where $r[i]$ is from random tapes, then computes $c_{\boldsymbol{r'}} \leftarrow$ PrivCmp.Enc($pk, \boldsymbol{r'}$).
3. $\mathrm{Sim}_{\mathrm{P}_2}^{\mathrm{cmp}}$ outputs $(b; c_{\boldsymbol{r'}})$.

We note that $\mathrm{View}_{\mathrm{P}_2}^{\Pi_{\mathrm{cmp}}}(\boldsymbol{a}, \boldsymbol{b}) = (\boldsymbol{b}; c_{\boldsymbol{a'}})$. Since $c_{\boldsymbol{r'}}$ and $c_{\boldsymbol{a'}}$ are indistinguishable due to the semantically secure of LHE, there exists a simulator such that $\mathrm{Sim}_{\mathrm{P}_2}^{\mathrm{cmp}}(\boldsymbol{b}) \equiv_c \mathrm{View}_{\mathrm{P}_2}^{\Pi_{\mathrm{cmp}}}(\boldsymbol{a}, \boldsymbol{b})$. □

Theorem 4: If the LHE is semantically secure, Algorithm 13 securely implements the ideal BPDTE functionality in the semi-honest model.

Proof the theorem 4: The correctness is guaranteed by Theorem 2. Let us construct the simulators $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{bpdte}}$ and $\mathrm{Sim}_{\mathrm{P}_2}^{\mathrm{bpdte}}$.

First, the simulator $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{bpdte}}$ has the public parameters λ, L, plus P_1's input $\boldsymbol{x} = [\boldsymbol{x}[\boldsymbol{0}], \ldots, \boldsymbol{x}[\boldsymbol{M-1}]]^T$ and output $\boldsymbol{\theta'} = [\mathcal{M}(\boldsymbol{x}[\boldsymbol{0}]), \ldots, \mathcal{M}(\boldsymbol{x}[\boldsymbol{M-1}])]$. Then:

1. $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{bpdte}}$ runs LHE.KeyGen($1^\lambda, 1^L$) to obtain pk, sk.

2. $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{bpdte}}$computes $\sigma[i]' \leftarrow \mathrm{PrivCmp.Encode}(\sigma[i])$ and stitching them one by one to obtain $\boldsymbol{\sigma'}$, where $\sigma[i] = \theta[i]'$.
3. Computes $c_{\boldsymbol{\sigma'}} \leftarrow \mathrm{LHE.Enc}(pk, \boldsymbol{\sigma'})$.
4. $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{bpdte}}$ outputs $(a; c_{\boldsymbol{\sigma'}})$.

We note that $\mathrm{View}_{\mathrm{P}_1}^{\Pi_{\mathrm{bpdte}}}(\mathcal{M}, \boldsymbol{x}) = (a; c_{\boldsymbol{\theta'}})$, where $c_{\boldsymbol{\sigma'}}$ and $c_{\boldsymbol{\theta'}}$ are indistinguishable due to the semantically secure of LHE. Furthermore, even if P_1 runs $\mathrm{LHE.Dec}(sk, c_{\boldsymbol{\theta'}})$ and $\mathrm{LHE.Dec}(sk, c_{\boldsymbol{\sigma'}})$ to obtain $\boldsymbol{\theta'}, \boldsymbol{\sigma'}$, they remain indistinguishable because they are the same value. Therefore, there exists a simulator such that $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{bpdte}}(\boldsymbol{x}, \boldsymbol{\theta'}) \equiv_c \mathrm{View}_{\mathrm{P}_1}^{\Pi_{\mathrm{bpdte}}}(\mathcal{M}, \boldsymbol{x})$.

Secondly, the simulator $\mathrm{Sim}_{\mathrm{P}_2}^{\mathrm{bpdte}}$ has the public parameters λ, L, along with P_2's input $\mathcal{M}$. Then:

1. $\mathrm{Sim}_{\mathrm{P}_2}^{\mathrm{bpdte}}$ runs $\mathrm{LHE.KeyGen}(1^\lambda, 1^L)$ to obtain pk.
2. $\mathrm{Sim}_{\mathrm{P}_2}^{\mathrm{bpdte}}$ computes $\mathrm{BPDTE.Encode}(\boldsymbol{r_x})$ to obtain $\boldsymbol{r_x}'$, where $\boldsymbol{r_x}$ is generated from random tapes. Then, it computes $c_{\boldsymbol{r_x}'} \leftarrow \mathrm{PrivCmp.Enc}(pk, \boldsymbol{r_x}')$.
3. $\mathrm{Sim}_{\mathrm{P}_2}^{\mathrm{bpdte}}$ outputs $(b; c_{\boldsymbol{r_a}'})$.

We note that $\mathrm{View}_{\mathrm{P}_2}^{\Pi_{\mathrm{bpdte}}}(\mathcal{M}, \boldsymbol{x}) = (b; c_{\boldsymbol{x'}})$, and $c_{\boldsymbol{r_x}'}$ and $c_{\boldsymbol{x'}}$ are indistinguishable due to the semantically secure of LHE. Therefore, there exists a simulator such that $\mathrm{Sim}_{\mathrm{P}_2}^{\mathrm{bpdte}}(\mathcal{M}) \equiv_c \mathrm{View}_{\mathrm{P}_2}^{\Pi_{\mathrm{bpdte}}}(\mathcal{M}, \boldsymbol{x})$. □

Theorem 5: Algorithm 14 securely implements the ideal CRR functionality in the semi-honest model.

Proof the theorem 5: Due to the analysis in Lemma 6, the correctness is holds. We construct a simulator $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{crr}}$ with P_1's input $\boldsymbol{x}, \boldsymbol{y}$ and output

$$x[i][j] \rightarrow_r x[i][\mathrm{P}_{\mathrm{L}}^{\mathrm{i}}(\mathrm{P}_{\mathrm{L}}(j))] \tag{48}$$

$$y[i][j] \rightarrow_r y[i][\mathrm{P}_{\mathrm{L}}^{\mathrm{i}}(\mathrm{P}_{\mathrm{L}}(j))] \tag{49}$$

where $\mathrm{P}_{\mathrm{L}}^{\mathrm{i}} \leftarrow_r \{\mathrm{P}_{\mathrm{L}}^1, \ldots, \mathrm{P}_{\mathrm{L}}^{\mathrm{M}}\}$. And any two $\mathrm{P}_{\mathrm{L}}^{\mathrm{i}}$ are same with probability $1/M$.

1. $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{crr}}$ randomly chooses a permutation f'.
2. $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{crr}}$ randomly chooses the permutation $\{\mathrm{g}_1', \ldots, \mathrm{g}_{\mathrm{M}}'\}$.
3. $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{crr}}$ computes that for all $i \in [M]$, $j \in [L]$.

$$(\boldsymbol{x}'[i][j], \boldsymbol{y}'[i][j]) \leftarrow (x[i][\mathrm{g}_{\mathrm{i}}'(\mathrm{f}'(j))], y[i][\mathrm{g}_{\mathrm{i}}'(\mathrm{f}'(j))]) \tag{50}$$

4. $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{crr}}$ outputs $(\boldsymbol{x}', \boldsymbol{y}')$.

We note that $\mathrm{View}_{\mathrm{P}_1}^{\Pi_{\mathrm{crr}}}((x[i][\mathrm{g}_{\mathrm{i}}(\mathrm{f}(j))], y[i][\mathrm{g}_{\mathrm{i}}(\mathrm{f}(j))]), i \in [M], j \in [L], \boldsymbol{x}, \boldsymbol{y}) = (\boldsymbol{x}', \boldsymbol{y}')$. Due to the same random process, the distributions between $\mathrm{Sim}_{\mathrm{P}_1}^{\mathrm{crr}}$ and $\mathrm{View}_{\mathrm{P}_1}^{\Pi_{\mathrm{crr}}}$ are indistinguishable. □

8 Performance of Ours Schemes

In this section, we present two benchmarks: PrivCMP and BPDTE. The device is a 13th Gen Intel(R) Core(TM) i7-13700KF 32GB of RAM, operating on a single thread.

8.1 Performance of PrivCMP

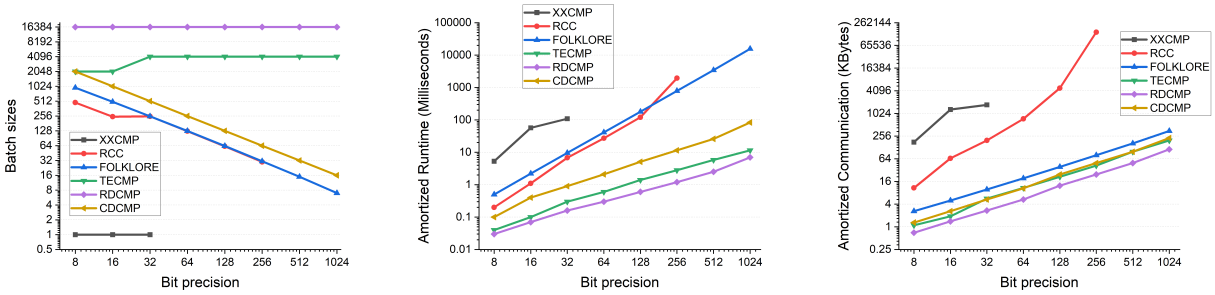


Fig. 5. Benchmark of PrivCMP, batch sizes, and amortized time (ms), amortized communication (KB)

In Figure 5, we banchmark the PrivCMP from 8 to 1024 bit precision. RDCMP exhibits the lowest amortized cost, CDCMP demonstrating a small and constant total communication cost. Meanwhile, TECMP reaches a bit precision of up to 26624 under a multiplicative depth of 12. At 32-bit precision, TECMP, RDCMP, and

Table 5. PrivCMP, the polynomial degree is N, and the h_n are 2, 4, 6, 8,..., satisfy that $\binom{l_n}{h_n} \geq 2^n$

Symbol	RCCMP	CWCMP
Bit precision	n	n
Mul. depth	$\lceil \log_2(h_n) \rceil$	$\lceil \log_2(\frac{(h_n+4)(h_n+1)}{2} + 1) \rceil$
Vector numbers	$\sum_{i=1}^{n} l_i$	l_n
Mul. numbers	$\sum_{i=1}^{n} h_i - n$	$\frac{(h_n+4)(h_n+1)}{2} + 1$
Input type	plain-cipher many to one	plain-cipher, many to one
Batch size	N	N

CDCMP are respectively 22×, 42×, and 7× faster, while achieving communication costs that are 65×, 73×, and 37× smaller than the RCC, not to mention the Folklore and XXCMP.

We theoretically analyze the performance differences between CWCMP and RCCMP from [15] in Table 5, that the former has more efficient performance than latter. Compared to our schemes in Table 4, at a bit precision of $n = 16$ (where $h_n = 4$ and $l_n \approx 27$), CWCMP achieves a batch size of 16,384, requiring 5 multiplicative depth, 27 ciphertexts, and 21 multiplications. In contrast, our RDCMP achieves a batch size of 16,383 with the same multiplicative depth but requires 16 ciphertexts and 30 multiplications. This demonstrates that RDCMP incurs slightly higher computational costs but achieves a lower communication costs compared to CWCMP. Moreover, our TECMP and CDCMP exhibit significantly better performance than CWCMP at smaller batch sizes.

8.2 Performance of BPDTE

We benchmark the decision tree trained on the UCI dataset [18] and present a comparison of our BPDTE with Sortinghat, Level Up and BPDTE_CW. In Table 6, the parameters are primarily include four components: the bit precision of the data, the number of columns in the data, and the number of internal nodes and leaf nodes in the decision tree.

Table 6. The characteristics of the trees trained on the UCI datasets.

UCI [18] name	Data		Tree	
	column	bit precision	internal	leaf
Heart	13	11	4	5
Breast	30	11	16	17
Spam	57	11	57	59
Electricity	8	10	553	554

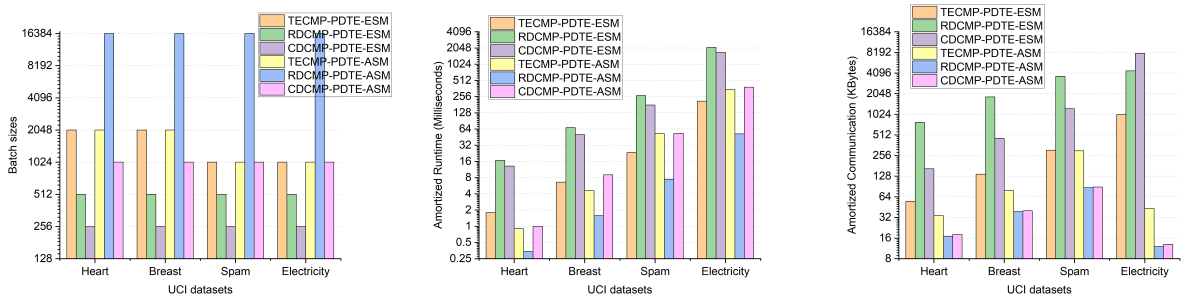


Fig. 6. Benchmark of BPDTE: batch sizes, amortized time (ms), and amortized communication (KB)

In Figure 6, Our RDCMP-PDTE-ASM demonstrates the best performance across various trees, with an amortized runtime lower than 60 ms. For small batch sizes, TECMP-PDTE-ESM, TECMP-PDTE-ASM, and CDCMP-PDTE-ASM achieve a balance between communication and computation costs. Notably, CDCMP-PDTE-ASM exhibits a relatively low overall communication cost, remaining under 100 MB for a batch size of 1024.

In Table 5, CWCMP demonstrates comparable performance to our RDCMP, meaning that BPDTE_CW in [26] achieves a runtime lower than our RDCMP-PDTE-ASM, but with higher communication costs. Meanwhile, the batch size of BPDTE_CW is fixed at N (e.g., 16,384), which makes its performance with smaller batch sizes less efficient than our schemes. In Table 7, we ran the Sortinghat and Level Up schemes on the same machines and same datasets. In term of electricity around 1000 nodes, XXCMP-PDTE has a better performance, at 2.945s and 548KB, followed by Sortinghat and RCC-PDTE. Compare with our schemes, the TECMP-PDTE-ESM and RDCMP-PDTE-ASM is 14× and 56× faster than the XXCMP-PDTE.

Table 7. The runtime (ms) and communication (KB) of Sortinghat [14] and Level Up [2]

	Sortinghat [14]			XXCMP-PDTE [2]			FOLKLORE-PDTE [2]			RCC-PDTE [2]		
Name	num.	time	comm.	num.	time	comm.	num.	time	comm.	num.	time	comm.
heart	1	38	8096	1	9	780	1	928	1568	1	200	7454
breast	1	154	18568	1	47	1570	1	1560	1568	1	405	7454
spam	1	543	35200	1	200	2824	1	3986	1568	1	1378	7454
electricity	1	5232	5016	1	2945	548	1	36791	1568	1	12225	6589

9 Conclusion

In this work, we propose three PrivCMP schemes: TECMP, RDCMP, and CDCMP. These schemes are non-interactive, achieving a logarithmic increase in bit precision relative to multiplicative depth. The key finding is that TE can simultaneously compute both the greater than and equal to comparisons between two numbers. Additionally, a new folklore-inspired dichotomy method is used, followed by batch encoding to complete our PrivCMP. This design enables our PrivCMP support various input types, Optional performance configurations, and flexible batch sizes.

We compared the computation and communication costs of several PrivCMP schemes. RDCMP has the lowest amortized runtime, while CDCMP has the lowest communication cost. TECMP offers a relatively moderate balance, with great computing speed, communication efficiency, and virtually unlimited bit precision. Compared to XXCMP, RCC, and FOLKLORE, RDCMP shows at least 10× faster speed and lower communication cost. Compared to CWCMP, RDCMP has a simpler structure, lower communication cost and more flexibility in batch size.

The security of SumPath in Level Up [2] is further enhanced during the batching process through the design of a new CRR algorithm. Building on our three PrivCMP schemes and two SumPath variants, we propose six BPDTE schemes that leverage the advantages of PrivCMP, resulting efficient performance. In large datasets, RDCMP-PDTE-ASM outperforms BPDTE_CW with lower communication costs. Compared to Sortinghat, XXCMP-PDTE, FOLKLORE-PDTE, and RCC-PDTE, our TECMP-PDTE-ESM and RDCMP-PDTE-ASM demonstrate improvements of at least 14× and 56× in computation speed, along with lower communication costs.

References

1. Akavia, A., Leibovich, M., Resheff, Y.S., Ron, R., Shahar, M., Vald, M.: Privacy-preserving decision trees training and prediction. ACM Transactions on Privacy and Security **25**(3), 1–30 (2022). https://doi.org/10.1145/3517197
2. Akhavan Mahdavi, R., Ni, H., Linkov, D., Kerschbaum, F.: Level up: Private non-interactive decision tree evaluation using levelled homomorphic encryption. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. pp. 2945–2958. CCS '23, Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3576915.3623095
3. Angel, S., Chen, H., Laine, K., Setty, S.: Pir with compressed queries and amortized query processing. In: 2018 IEEE Symposium on Security and Privacy. pp. 962–979. SP '18, Institute of Electrical and Electronics Engineers, San Francisco, CA, USA (2018). https://doi.org/10.1109/SP.2018.00062
4. Azar, A.T., El-Metwally, S.M.: Decision tree classifiers for automated medical diagnosis. Neural Computing and Applications **23**(7), 1433–3058 (2013). https://doi.org/10.1007/s00521-012-1196-7
5. Azogagh, S., Delfour, V., Gambs, S., Killijian, M.O.: Probonite: Private one-branch-only non-interactive decision tree evaluation. In: Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography. pp. 23–33. WAHC'22, Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3560827.3563377
6. Bai, J.L., Song, X.F., Cui, S.J., Chang, E.C., Russello, G.: Scalable private decision tree evaluation with sublinear communication. In: Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security. pp. 843–857. ASIA CCS '22, Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3488932.3517413
7. Brakerski, Z., Vaikuntanathan, V.: Efficient fully homomorphic encryption from (standard) lwe. SIAM Journal on Computing **43**(2), 831–871 (2014). https://doi.org/10.1137/120868669
8. Brickell, J., Porter, D.E., Shmatikov, V., Witchel, E.: Privacy-preserving remote diagnostics. In: Proceedings of the 14th ACM Conference on Computer and Communications Security. pp. 498–507. CCS '07, Association for Computing Machinery, New York, NY, USA (2007). https://doi.org/10.1145/1315245.1315307
9. Buckman, J., Roy, A., Raffel, C., Goodfellow, I.: Thermometer encoding: One hot way to resist adversarial examples. In: International Conference on Learning Representations. ICLR '18 (2018)
10. Chen, H., Chillotti, I., Ren, L.: Onion ring oram: Efficient constant bandwidth oblivious ram from (leveled) tfhe. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 345–360. CCS '19, Association for Computing Machinery, New York, NY, USA (2019). https://doi.org/10.1145/3319535.3354226
11. Cheon, J.H., Kim, M., Kim, M.: Search-and-compute on encrypted data. In: Financial Cryptography and Data Security. FC '15, vol. 8976, pp. 142–159. Springer Berlin Heidelberg, Berlin, Heidelberg (2015). https://doi.org/10.1007/978-3-662-48051-9_11
12. Cheon, J.H., Kim, M., Lauter, K.: Homomorphic computation of edit distance. In: Financial Cryptography and Data Security. FC '15, vol. 8976, pp. 194–212. Springer Berlin Heidelberg, Berlin, Heidelberg (2015). https://doi.org/10.1007/978-3-662-48051-9_15

13. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Tfhe: Fast fully homomorphic encryption over the torus. Journal of Cryptology **33**, 57 (2020). https://doi.org/https://doi.org/10.1007/s00145-019-09319-x
14. Cong, K., Das, D., Park, J., Pereira, H.V.: Sortinghat: Efficient private decision tree evaluation via homomorphic encryption and transciphering. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. pp. 563–577. CCS '22, Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3548606.3560702
15. Cong, K., Kang, J., Nicolas, G., Park, J.: Faster private decision tree evaluation for batched input from homomorphic encryption. In: Security and Cryptography for Networks. pp. 3–23. SCN '24, Springer Nature Switzerland, Cham (2024)
16. Damgård, I., Geisler, M., Krøigaard, M.: Efficient and secure comparison for on-line auctions. In: Information Security and Privacy. ACISP '07, vol. 4586, pp. 416–430. Springer Berlin Heidelberg, Berlin, Heidelberg (2007). https://doi.org/10.1007/978-3-540-73458-1_30
17. Damgard, I., Geisler, M., Kroigard, M.: A correction to 'efficient and secure comparison for on-line auctions'. Int. J. Appl. Cryptol. **1**(4), 323–324 (2009). https://doi.org/10.1504/IJACT.2009.028031
18. Dua, D., Graff, C., et al.: Uci machine learning repository (2017)
19. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive **2012**(144), 1–19 (2012)
20. Gentry, C., Halevi, S., Jutla, C., Raykova, M.: Private database access with he-over-oram architecture. In: Applied Cryptography and Network Security. ACNS '15, vol. 9092, pp. 172–191. Springer International Publishing, Cham (2015). https://doi.org/10.1007/978-3-319-28166-7_9
21. Kiayias, A., Papadopoulos, S., Triandopoulos, N., Zacharias, T.: Delegatable pseudorandom functions and applications. In: Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security. pp. 669–684. CCS '13, Association for Computing Machinery, New York, NY, USA (2013). https://doi.org/10.1145/2508859.2516668
22. Kim, A., Polyakov, Y., Zucca, V.: Revisiting homomorphic encryption schemes for finite fields. In: Advances in Cryptology. ASIACRYPT '21, vol. 13092, pp. 608–639. Springer International Publishing, Cham (2021). https://doi.org/10.1007/978-3-030-92078-4_21
23. Kiss, Á., Naderpour, M., Liu, J., Asokan, N., Schneider, T.: Sok: Modular and efficient private decision tree evaluation. Proceedings on Privacy Enhancing Technologies **2019**(2), 187–208 (2019). https://doi.org/10.2478/popets-2019-0026
24. Lin, H.Y., Tzeng, W.G.: An efficient solution to the millionaires' problem based on homomorphic encryption. In: Applied Cryptography and Network Security. ACNS '05, vol. 3531, pp. 456–466. Springer Berlin Heidelberg, Berlin, Heidelberg (2005). https://doi.org/10.1007/11496137_31
25. Lindell, Y.: How to simulate it - a tutorial on the simulation proof technique. Cryptology ePrint Archive **2016**(046), 1–65 (2016)
26. Liu, K., Tang, C.M., et al.: Secure two-party decision tree classification based on function secret sharing. Complexity **2023**, 1–13 (2023). https://doi.org/10.1155/2023/5302915
27. Lu, W.J., Huang, Z.C., Zhang, Q.Z., Wang, Y.C., Hong, C.: Squirrel: A scalable secure Two-Party computation framework for training gradient boosting decision tree. In: 32nd USENIX Security Symposium. pp. 6435–6451. USENIX Security '23, USENIX Association, Anaheim, CA (2023)
28. Lu, W.J., Zhou, J.J., Jun, S.: Non-interactive and output expressive private comparison from homomorphic encryption. In: Proceedings of the 2018 on Asia Conference on Computer and Communications Security. pp. 67–74. ASIACCS '18, Association for Computing Machinery, New York, NY, USA (2018). https://doi.org/10.1145/3196494.3196503
29. Nateghizad, M., Erkin, Z., Lagendijk, R.L.: An efficient privacy-preserving comparison protocol in smart metering systems. EURASIP Journal on Information Security **2016**(11), 1–8 (2016). https://doi.org/10.1186/s13635-016-0033-4
30. Papernot, N., McDaniel, P., Goodfellow, I., Jha, S., Celik, Z.B., Swami, A.: Practical black-box attacks against machine learning. In: Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security. pp. 506–519. ASIA CCS '17, Association for Computing Machinery, New York, NY, USA (2017). https://doi.org/10.1145/3052973.3053009
31. Raphael, B., Raluca, A.P., Stephen, T., Shafi, G.: Machine learning classification over encrypted data. Cryptology ePrint Archive **2014**(331), 1–34 (2015)
32. Rasoul, A.M., Florian, K.: Constant-weight PIR: Single-round keyword PIR via constant-weight equality operators. In: 31st USENIX Security Symposium. pp. 1723–1740. USENIX Security '22, USENIX Association, Boston, MA (2022)
33. Rokach, L., Maimon, O.: Top-down induction of decision trees classifiers - a survey. IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews) **35**(4), 476–487 (2005). https://doi.org/10.1109/TSMCC.2004.843247
34. Shi, E., Bethencourt, J., Chan, T.H.H., Song, D., Perrig, A.: Multi-dimensional range query over encrypted data. In: 2007 IEEE Symposium on Security and Privacy. pp. 350–364. SP '07, Institute of Electrical and Electronics Engineers, Berkeley, CA, USA (2007). https://doi.org/10.1109/SP.2007.29
35. Singh, A., Guttag, J.V.: A comparison of non-symmetric entropy-based classification trees and support vector machine for cardiovascular risk stratification. In: 2011 Annual International Conference of the IEEE Engineering in Medicine and Biology Society. pp. 79–82. IEEE, Boston, MA, USA (2011). https://doi.org/10.1109/IEMBS.2011.6089901
36. Tai, R.K.H., Ma, J.P.K., Zhao, Y.J., Chow, S.S.M.: Privacy-preserving decision trees evaluation via linear functions. In: Computer Security – ESORICS 2017. ESORICS '17, vol. 10493, pp. 494–512. Springer International Publishing, Cham (2017). https://doi.org/10.1007/978-3-319-66399-9_27
37. Tueno, A., Boev, Y., Kerschbaum, F.: Non-interactive private decision tree evaluation. In: Data and Applications Security and Privacy XXXIV. DBsec '20, vol. 12122, pp. 174–194. Springer International Publishing, Cham (2020). https://doi.org/10.1007/978-3-030-49669-2_10

38. Tueno, A., Kerschbaum, F., Katzenbeisser, S.: Private evaluation of decision trees using sublinear cost. Proceedings on Privacy Enhancing Technologies **2019**(1), 266–286 (2019). https://doi.org/10.2478/popets-2019-0015
39. Veugen, T.: Improving the dgk comparison protocol. In: 2012 IEEE International Workshop on Information Forensics and Security (WIFS). pp. 49–54. IEEE, Costa Adeje, Spain (2012). https://doi.org/10.1109/WIFS.2012.6412624
40. Witten, I.H., Frank, E., Hall, M.A.: Data Mining: Practical Machine Learning Tools and Techniques. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 3rd edn. (2011)
41. Wu, D.J., Feng, T., Naehrig, M., Lauter, K.: Privately evaluating decision trees and random forests. Proceedings on Privacy Enhancing Technologies **2016**(4), 335–355 (2016). https://doi.org/10.1515/popets-2016-0043
42. Yao, A.C.: Protocols for secure computations. In: 23rd Annual Symposium on Foundations of Computer Science (sfcs 1982). pp. 160–164. Chicago, IL, USA (1982). https://doi.org/10.1109/SFCS.1982.38
43. Yu, H., Zhang, H., Jia, X., Chen, X., Yu, X.: psafety: Privacy-preserving safety monitoring in online ride hailing services. IEEE Transactions on Dependable and Secure Computing **20**(1), 209–224 (2023). https://doi.org/10.1109/TDSC.2021.3130571
44. Zhang, M., Chen, Y., Susilo, W.: Decision tree evaluation on sensitive datasets for secure e-healthcare systems. IEEE Transactions on Dependable and Secure Computing **20**(5), 3988–4001 (2023). https://doi.org/10.1109/TDSC.2022.3219849
45. Zhang, Z., Zhang, H., Song, X., Lin, J., Kong, F.: Secure outsourcing evaluation for sparse decision trees. IEEE Transactions on Dependable and Secure Computing **21**(6), 5228–5241 (2024). https://doi.org/10.1109/TDSC.2024.3372505
46. Zheng, Y., Zhu, H., Lu, R., Guan, Y., Zhang, S., Wang, F., Shao, J., Li, H.: Phrknn: Efficient and privacy-preserving reverse knn query over high-dimensional data in cloud. IEEE Transactions on Dependable and Secure Computing **21**(4), 1831–1844 (2024). https://doi.org/10.1109/TDSC.2023.3291715